

Họ và tên: Vũ Đại Dương

MSSV: 23520359

Lớp: IT007.P28

MÔN HỌC: HỆ ĐIỀU HÀNH
CÂU HỎI VÀ BÀI TẬP CHƯƠNG 6

1. Deadlock là gì?

- Một tiến trình gọi là deadlock nếu nó đang đợi một sự kiện mà sẽ không bao giờ xảy ra.

2. Các điều kiện cần để xảy ra deadlock?

- Loại trừ tương hỗ: ít nhất 1 tài nguyên được giữ theo nonsharable mode
- Giữ và chờ cấp thêm tài nguyên: 1 tiến trình đang giữ ít nhất 1 tài nguyên và đợi thêm tài nguyên do tiến trình khác giữ.
- Không trung dụng: tài nguyên không thể bị lấy lại mà chỉ có thể được trả lại từ tiến trình đang giữ tài nguyên đó khi nó muốn.
- Chu trình đợi: tồn tại 1 tập $\{P_0, \dots, P_n\}$ các tiến trình đang đợi sao cho:
 - P_0 đợi 1 tài nguyên mà P_1 giữ
 - P_1 đợi 1 tài nguyên mà P_2 giữ
 - ...
 - P_n đợi 1 tài nguyên mà P_0 giữ

3. Đồ thị cấp phát tài nguyên là gì? Mối liên hệ giữa đồ thị cấp phát tài nguyên và deadlock?

- Đồ thị cấp phát tài nguyên là đồ thị có hướng, với tập đỉnh V và tập cạnh E
 - Tập đỉnh V gồm 2 loại:
 - $P = \{P_1, P_2, \dots, P_n\}$ (All process)

- $R = \{R_1, R_2, \dots, R_n\}$ (All resource)
- Tập cạnh E gồm 2 loại:
 - Cạnh yêu cầu: $P_i \rightarrow R_j$
 - Cạnh cấp phát: $R_j \rightarrow P_i$
- Mỗi liên hệ giữ đồ thị cấp phát tài nguyên và deadlock:
 - RAG không chứa chu trình $\Rightarrow$ không có deadlock
 - RAG chứa 1 (hay nhiều) chu trình:
 - Nếu mỗi loại tài nguyên chỉ có 1 thực thể $\Rightarrow$ deadlock
 - Nếu mỗi loại tài nguyên có nhiều thực thể $\Rightarrow$ có thể xảy ra deadlock

4. Có mấy phương pháp để giải quyết deadlock? Phân tích và đánh giá ưu, nhược điểm của từng phương pháp?

- Có 4 phương pháp để giải quyết deadlock:
 - Ngăn deadlock: không cho phép (ít nhất) 1 trong 4 điều kiện cần cho deadlock
 - Tránh deadlock: các tiến trình cần cung cấp thông tin về tài nguyên nó cần để hệ thống cấp phát tài nguyên 1 cách thích hợp
 - Cho phép hệ thống vào trạng thái deadlock, nhưng sau đó phát hiện và phục hồi hệ thống
 - Bỏ qua mọi vấn đề, xem như deadlock không bao giờ xảy ra trong hệ thống

	Ưu điểm	Nhược điểm
Ngăn deadlock	- Đảm bảo hệ thống không bao giờ bị	- Thường gây lãng phí tài nguyên hoặc giảm

	deadlock - Dễ áp dụng cho hệ thống đơn giản - Không cần cơ chế phát hiện hay phục hồi	hiệu suất - Khó thực hiện trong hệ thống thực tế phức tạp - Có thể gây nghẽn tài nguyên giả tạo
Tránh deadlock	- Đảm bảo hệ thống luôn trong trạng thái an toàn - Tối ưu tài nguyên hơn so với phương pháp ngăn deadlock - Không cần phát hiện/phục hồi	- Cần tiến trình khai báo trước trước tài nguyên tối đa cần dùng - Tốn thời gian tính toán trước mỗi lần cấp phát - Không phù hợp nếu hệ thống không thể biết trước nhu cầu tài nguyên
Phát hiện và phục hồi	- Cho phép sử dụng tài nguyên hiệu quả, linh hoạt - Phù hợp với hệ thống không thể tránh deadlock	- Cần thuật toán kiểm tra deadlock liên tục - Tốn chi phí phục hồi - Phục hồi có thể gây rủi ro cho tiến trình đang chạy
Bỏ qua	- Đơn giản, không tốn	- Deadlock không được

	chi phí xử lý - Phù hợp cho hệ thống không quan trọng hoặc ít tiến trình	phát hiện, dẫn đến giảm hiệu suất của hệ thống → Hệ thống có thể ngưng hoạt động và phải khởi động lại
--	---	--

5. Phân tích và đánh giá ưu, nhược điểm của các giải pháp đồng bộ busy waiting (cả phần cứng và phần mềm)?

- Giải pháp đồng bộ busy waiting là các kỹ thuật đồng bộ không nhường CPU khi chờ, mà tiến trình sẽ quay vòng cho đến khi điều kiện đồng bộ thỏa mãn

		Ưu điểm	Nhược điểm
Giải pháp phần mềm	Dùng flag	- Dễ hiểu, đơn giản, không cần phần cứng hỗ trợ	- Không đảm bảo mutual exclusion - Dễ gặp race condition - Không mở rộng cho nhiều tiến trình
	Peterson	- Đảm bảo mutual exclusion, progress, bounded waiting	- Chỉ dùng cho 2 tiến trình - Tốn CPU vì bận liên

		- Dừng biến đơn giản, không cần phần cứng	tục
Giải pháp phần cứng	Tắt ngắt (Disable interrupts)	- Đảm bảo mutual exclusion	- Chỉ dùng được trên hệ thống đơn CPU - Không công bằng, có thể gây treo hệ thống
	Lệnh nguyên tử (Test-and-Set, Swap, Compare-and-Swap)	- Đảm bảo an toàn cho nhiều tiến trình - Dừng được cho đa CPU	- Tốn CPU - Có thể starvation

6. Trạng thái an toàn là gì? Mối liên hệ giữa trạng thái an toàn và deadlock?

- 1 trạng thái của hệ thống được gọi là an toàn (safe) nếu tồn tại 1 chuỗi thứ tự an toàn
- Nếu hệ thống đang ở trạng thái safe => không deadlock

7. Mô tả cách thực hiện các giải thuật Banker: giải thuật an toàn, giải thuật yêu cầu tài nguyên và giải thuật phát hiện deadlock?

- Giải thuật an toàn:
 - o Gọi Work và Finish là 2 vector độ dài là m và n. Khởi tạo Work = Available;
 $Finish[i] = false, i = 0, 1, \dots, n - 1$

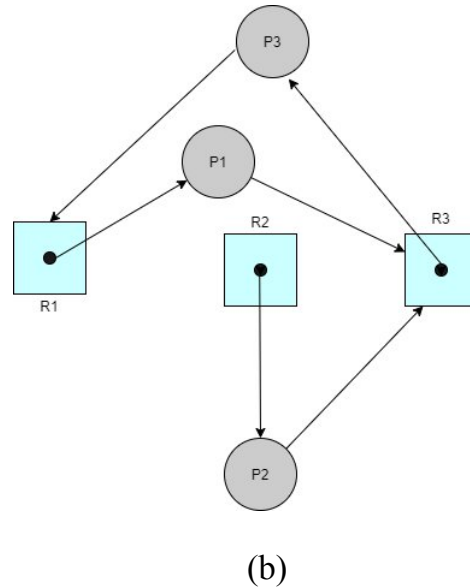
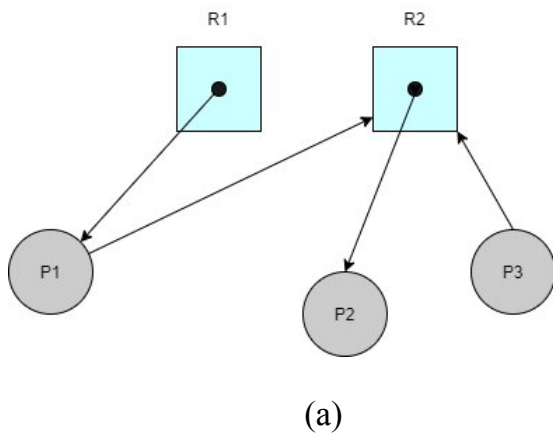
- Tìm i thỏa: $Finish[i] = false$, $Need_i \leq Work$ (hàng thứ i của Need). Nếu không tồn tại i như vậy, đến bước 4
- $Work = Work + Allocation_i$
 $Finish[i] = true$
 Quay về bước 2
- Nếu $Finish[i] = true$, $i = 1, \dots, n$, thì hệ thống đang ở trạng thái safe
- Giải thuật yêu cầu tài nguyên:
 - Input: $Request_i[j] = k$: P_i cần k instance của tài nguyên R_j
 - $Request_i > Need_i \Rightarrow$ báo lỗi tiến trình đã vượt yêu cầu tối đa
 - $Request_i > Available \Rightarrow$ chờ vì tài nguyên không đủ
 - Giả định cấp phát tài nguyên:
 - $Available = Available - Request_i$
 - $Allocation_i = Allocation_i + Request_i$
 - $Need_i = Need_i - Request_i$
 - Safe $\Rightarrow$ tài nguyên được cấp thực sự cho P_i ;
 Unsafe $\Rightarrow$ đợi và phục hồi trạng thái
- Giải thuật phát hiện deadlock:
 - Input: $Available[]$: số thực thể sẵn sàng của mỗi loại tài nguyên
 $Allocation[n][m]$: số thực thể của mỗi loại tài nguyên đã cấp phát cho mỗi tiến trình
 $Request[n][m]$: yêu cầu hiện tại của mỗi tiến trình
 - $Work = Available$

- Nếu $\text{Allocation}_i \neq 0 \Rightarrow \text{Finish}[i] := \text{false}$, ngược lại true
- Tìm i thỏa:
 - $\text{Finish}[i] = \text{false}$
 - $\text{Request}_i \leq \text{Work}$
 - Nếu tìm thấy:
 - $\text{Work} = \text{Work} + \text{Allocation}_i$
 - $\text{Finish}[i] = \text{true}$
 - Quay về bước 2
- Sau khi kết thúc: Các tiến trình có $\text{Finish}[i] = \text{false} \Rightarrow \text{deadlock}$

8. Nêu các giải pháp để phục hồi hệ thống sau khi phát hiện có deadlock?

- Khi deadlock xảy ra, để phục hồi:
 - Báo người vận hành
 - Hệ thống tự động phục hồi bằng cách bỏ gậy chu trình deadlock:
 - Chấm dứt 1 hay nhiều tiến trình.
 - Lấy lại tài nguyên từ 1 hay nhiều tiến trình.

9. (Bài tập mẫu) Cho các đồ thị cấp phát tài nguyên sau. Hỏi đồ thị nào có deadlock xảy ra?



Trả lời:

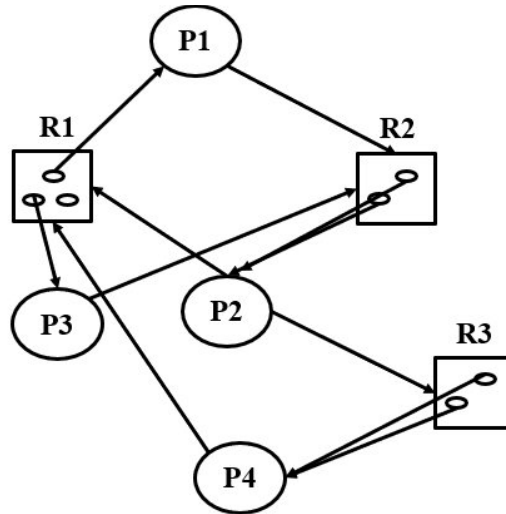
- Đồ thị (a) không có deadlock, do đồ thị này có chuỗi an toàn là: $\langle P2, P1, P3 \rangle$ hoặc $\langle P2, P3, P1 \rangle$.
- Đồ thị (b) có deadlock.

10. (Bài tập mẫu) Cho 1 hệ thống có 4 tiến trình P1, P2, P3, P4 và 3 loại tài nguyên R1 (3), R2 (2) R3 (2). P1 giữ 1 R1 và yêu cầu 1 R2; P2 giữ 2 R2 và yêu cầu 1 R1 và 1 R3; P3 giữ 1 R1 và yêu cầu 1 R2; P4 giữ 2 R3 và yêu cầu 1 R1.

- a. Vẽ đồ thị cấp phát tài nguyên của hệ thống
- b. Hệ thống có deadlock không?
- c. Tìm chuỗi an toàn (nếu có)

Trả lời:

- a. Đồ thị cấp phát tài nguyên



b. Hệ thống không bị deadlock do hệ thống có chuỗi an toàn.

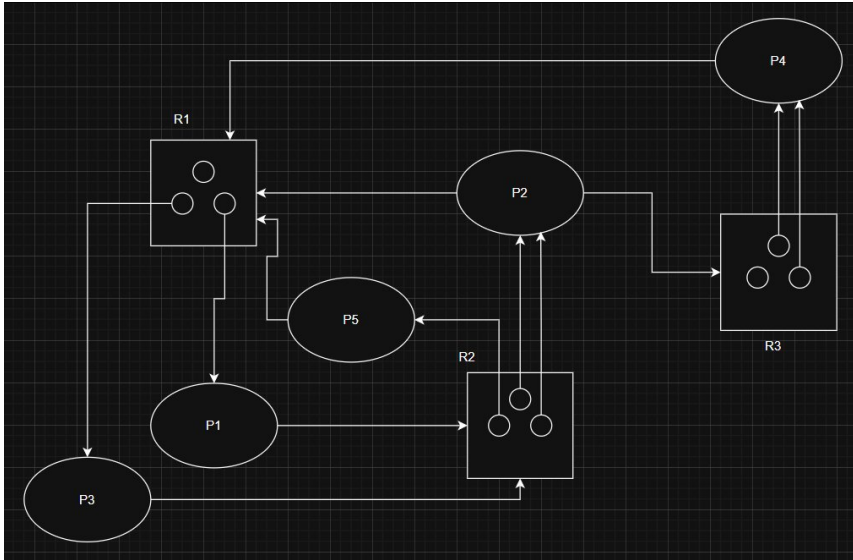
c. Chuỗi an toàn là: $\langle P4, P2, P3, P1 \rangle$ hoặc $\langle P4, P2, P1, P3 \rangle$.

11. Cho 1 hệ thống có 5 tiến trình P1, P2, P3, P4, P5 và 3 loại tài nguyên R1 (có 3 thực thể), R2 (có 3 thực thể) R3 (có 2 thực thể). P1 giữ 1 thực thể R1 và yêu cầu 1 thực thể R2; P2 giữ 2 thực thể R2 và yêu cầu 1 thực thể R1 và 1 thực thể R3; P3 giữ 1 thực thể R1 và yêu cầu 1 thực thể R2; P4 giữ 2 thực thể R3 và yêu cầu 1 thực thể R1; P5 đang giữ 1 thực thể của R2 và yêu cầu 1 thực thể của R1.

- Vẽ đồ thị cấp phát tài nguyên
- Có bao nhiêu chuỗi an toàn cho hệ thống trên?
- Viết ra tất cả các chuỗi an toàn (nếu có)

Trả lời:

- Đồ thị cấp phát tài nguyên



b. Số chuỗi an toàn trên hệ thống: 6 chuỗi

- c.
- $\langle P4, P2, P1, P3, P5 \rangle$
 - $\langle P4, P2, P1, P5, P3 \rangle$
 - $\langle P4, P2, P3, P1, P5 \rangle$
 - $\langle P4, P2, P3, P5, P1 \rangle$
 - $\langle P4, P2, P5, P1, P3 \rangle$
 - $\langle P4, P2, P5, P3, P1 \rangle$

12. (Bài tập mẫu) Xét một hệ thống máy tính có 5 tiến trình: P0, P1, P2, P3, P4 và 4 loại tài nguyên: A, B, C, D. Tại thời điểm t_0 , trạng thái của hệ thống như sau:

Tiến trình	Allocation				Max			
	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2
P1	1	0	0	0	1	7	5	0
P2	1	3	5	4	2	3	5	6
P3	0	6	3	2	0	6	5	2
P4	0	0	1	4	0	6	5	6

Available			
A	B	C	D
1	5	2	0

- a. Tìm Need?
- b. Hệ thống có an toàn không?
- c. Nếu P1 yêu cầu (0,4,2,0) thì có thể cấp phát cho nó ngay không?

Trả lời:

a. Ma trận Need

	Need			
Tiến trình	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

b. Thực hiện giải thuật an toàn

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	0	0	1	2	0	0	1	2	0	0	0	0	1	5	2	0	P0
P1	1	0	0	0	1	7	5	0	0	7	5	0	1	5	3	2	P2
P2	1	3	5	4	2	3	5	6	1	0	0	2	2	8	8	6	P3
P3	0	6	3	2	0	6	5	2	0	0	2	0	2	14	11	8	P4
P4	0	0	1	4	0	6	5	6	0	6	4	2	2	14	12	12	P1

Hệ thống có chuỗi an toàn <P0, P2, P3, P4, P1> cho nên hệ thống an toàn.

c.

Request P1 (0,4,2,0) ≤ Need P1 (0, 7, 5, 0).

Request P1 (0,4,2,0) ≤ Available (1, 5, 2, 0).

Giả sử hệ thống đáp ứng yêu cầu (0,4,2,0) của P1.

Trạng thái mới của hệ thống:

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	0	0	1	2	0	0	1	2	0	0	0	0	1	1	0	0	P0
P1	1	4	2	0	1	7	5	0	0	3	3	0	1	1	1	2	P2
P2	1	3	5	4	2	3	5	6	1	0	0	2	2	4	6	6	P3
P3	0	6	3	2	0	6	5	2	0	0	2	0	2	10	9	8	P4
P4	0	0	1	4	0	6	5	6	0	6	4	2	2	10	7	12	P1

Hệ thống mới vẫn có chuỗi an toàn <P0, P2, P3, P4, P1> cho nên hệ thống đáp ứng yêu cầu cấp phát cho P1.

13. Xét hệ thống tại thời điểm t_0 có 5 tiến trình: P1, P2, P3, P4, P5; và 4 loại tài nguyên: R1, R2, R3, R4. Xét trạng thái hệ thống như sau:

Process	Allocation				Max			
	R1	R2	R3	R4	R1	R2	R3	R4
P1	0	0	1	2	0	0	3	2
P2	2	0	0	0	2	7	5	0
P3	0	0	3	4	6	6	5	6
P4	2	3	5	4	3	3	5	6
P5	0	3	3	2	0	6	5	2

Available			
R1	R2	R3	R4
2	1	2	0

- Tại thời điểm t_0 , áp dụng giải thuật banker tìm chuỗi an toàn của hệ thống?
- Tại thời điểm t_1 , tiến trình P3 yêu cầu thêm tài nguyên (1, 1, 0, 0) thì hệ thống có thể đáp ứng ngay được không? Tại sao?

Trả lời:

- Thực hiện giải thuật an toàn:

	Allocation				Max				Need				Available (Work)				
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	
P1	0	0	1	2	0	0	3	2	0	0	2	0	2	1	2	0	P1
P2	2	0	0	0	2	7	5	0	0	7	5	0	2	1	3	2	P4
P3	0	0	3	4	6	6	5	6	6	6	2	2	4	4	8	6	P5
P4	2	3	5	4	3	3	5	6	1	0	0	2	4	7	11	8	P2
P5	0	3	3	2	0	6	5	2	0	3	2	0	6	7	11	8	P3

Chuỗi an toàn của hệ thống: <P1, P4, P5, P2, P3>

- Request P3 (1, 1, 0, 0) $\leq$ Need P3 (6, 6, 2, 2)

Request P3 (1, 1, 0, 0) $\leq$ Available (2, 1, 2, 0)

Giả sử hệ thống đáp ứng yêu cầu (1, 1, 0, 0) của P3

Trạng thái mới của hệ thống:

	Allocation				Max				Need				Available (Work)				
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4	
P1	0	0	1	2	0	0	3	2	0	0	2	0	1	0	2	0	P1
P2	2	0	0	0	2	7	5	0	0	7	5	0	1	0	3	2	P4
P3	1	1	3	4	6	6	5	6	5	5	2	2	3	3	8	6	P5
P4	2	3	5	4	3	3	5	6	1	0	0	2	3	6	11	8	
P5	0	3	3	2	0	6	5	2	0	3	2	0					

Trạng thái mới là unsafe, vậy không thể cấp phát tài nguyên cho P3

14. Sử dụng giải thuật Banker để kiểm tra các trạng thái sau có an toàn hay không? Nếu có thì đưa ra chuỗi thực thi an toàn, nếu không thì nêu rõ lý do không an toàn?

a. Available = (0,3,0,1)

b. Available = (1,0,0,2)

	Allocation				Max			
Tiến trình	A	B	C	D	A	B	C	D
P0	3	0	1	4	5	1	1	7
P1	2	2	1	0	3	2	1	1
P2	3	1	2	1	3	3	2	1
P3	0	5	1	0	4	6	1	2
P4	4	2	1	2	6	3	2	5

Trả lời:

a. Thực hiện giải thuật an toàn:

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	3	0	1	4	5	1	1	7	2	1	0	3	0	3	0	1	P2
P1	2	2	1	0	3	2	1	1	1	0	0	1	3	4	2	2	P1
P2	3	1	2	1	3	3	2	1	0	2	0	0	5	6	3	2	P3
P3	0	5	1	0	4	6	1	2	4	1	0	2	5	11	4	2	P4
P4	4	2	1	2	6	3	2	5	2	1	1	3	9	13	5	4	P0

Chuỗi an toàn: <P2, P1, P3, P4, P0> => an toàn

b. Thực hiện giải thuật an toàn:

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	3	0	1	4	5	1	1	7	2	1	0	3	1	0	0	2	P1
P1	2	2	1	0	3	2	1	1	1	0	0	1	3	2	1	2	P2
P2	3	1	2	1	3	3	2	1	0	2	0	0	6	3	3	3	P0
P3	0	5	1	0	4	6	1	2	4	1	0	2	9	3	4	7	P3
P4	4	2	1	2	6	3	2	5	2	1	1	3	9	8	5	7	P4

Chuỗi an toàn: <P1, P2, P0, P3, P4> => an toàn

15. Trả lời các câu hỏi sau bằng cách sử dụng giải thuật Banker:

- Hệ thống có an toàn không? Đưa ra chuỗi an toàn nếu có?
- Nếu P1 yêu cầu (1,1,0,0) thì có thể cấp phát cho nó ngay không?
- Nếu P4 yêu cầu (0,0,2,0) thì có thể cấp phát cho nó ngay không?

Tiến trình	Allocation				Max			
	A	B	C	D	A	B	C	D
P0	2	0	0	1	4	2	1	2
P1	3	1	2	1	5	2	5	2
P2	2	1	0	3	2	3	1	6
P3	1	3	1	2	1	4	2	4
P4	1	4	3	2	3	6	6	5

Available			
A	B	C	D
3	3	2	1

Trả lời:

- Thực hiện giải thuật an toàn:

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	2	0	0	1	4	2	1	2	2	2	1	1	3	3	2	1	P0
P1	3	1	2	1	5	2	5	2	2	1	3	1	5	3	2	2	P3
P2	2	1	0	3	2	3	1	6	0	2	1	3	6	6	3	4	P1
P3	1	3	1	2	1	4	2	4	0	1	1	2	9	7	5	5	P2
P4	1	4	3	2	3	6	6	5	2	2	3	3	11	8	5	8	P4

Chuỗi an toàn của hệ thống: <P0, P3, P1, P2, P4> => Hệ thống an toàn

- Request P1 (1, 1, 0, 0) $\leq$ Need P1 (2, 2, 1, 1)
 Request P1 (1, 1, 0, 0) $\leq$ Available (3, 3, 2, 1)
 Giả sử hệ thống đáp ứng yêu cầu (1, 1, 0, 0) của P1
 Trạng thái mới của hệ thống:

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	2	0	0	1	4	2	1	2	2	2	1	1	2	2	2	1	P0
P1	4	2	2	1	5	2	5	2	1	0	3	1	4	2	2	2	P3
P2	2	1	0	3	2	3	1	6	0	2	1	3	5	5	3	4	P1
P3	1	3	1	2	1	4	2	4	0	1	1	2	9	7	5	5	P2
P4	1	4	3	2	3	6	6	5	2	2	3	3	11	8	5	8	P4

Hệ thống mới vẫn có chuỗi an toàn <P0, P3, P1, P2, P4> cho nên hệ thống đáp ứng yêu cầu cấp phát cho P1

c. Request P4 (0, 0, 2, 0) $\leq$ Need P4 (2, 2, 3, 3)

Request P4 (0, 0, 2, 0) $\leq$ Available (3, 3, 2, 1)

Giả sử hệ thống đáp ứng yêu cầu (0, 0, 2, 0) của P4

Trạng thái mới của hệ thống:

	Allocation				Max				Need				Available (Work)				
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D	
P0	2	0	0	1	4	2	1	2	2	2	1	1	3	3	0	1	
P1	3	1	2	1	5	2	5	2	2	1	3	1					
P2	2	1	0	3	2	3	1	6	0	2	1	3					
P3	1	3	1	2	1	4	2	4	0	1	1	2					
P4	1	4	5	2	3	6	6	5	2	2	1	3					

Hệ thống mới không an toàn, cho nên hệ thống không đáp ứng yêu cầu cấp phát cho P4